terraform_cheatsheet

Terraform describes infrastructure as code: init downloads providers, plan shows what will change, apply makes it so, and the state file remembers what exists. These are the commands you run every day, grouped by task. OpenTofu accepts the same commands: switch to tofu or a tf alias and every command updates. Click a command to copy it.

:workflow — 7 cmds

Init, plan, apply, destroy.

Download providers and set up the backend

```
$ tf init
```

Show what will change

```
$ tf plan
```

Apply the changes

```
$ tf apply
```

Apply without the confirmation prompt

```
$ tf apply -auto-approve
```

Save a plan, then apply exactly that plan

```
$ tf plan -out=tfplan   $ tf apply tfplan
```

Preview a destroy

```
$ tf plan -destroy
```

Destroy everything in the configuration

```
$ tf destroy
```

:plan — 8 cmds

Variables, targets and replacements.

Set a variable

```
$ tf plan -var='region=us-east-1'
```

Use a variables file

```
$ tf plan -var-file=prod.tfvars
```

Plan only one resource and its dependencies

```
$ tf plan -target=aws_instance.web
```

Force a resource to be recreated

```
$ tf apply -replace=aws_instance.web
```

Sync state with real infrastructure only

```
$ tf apply -refresh-only
```

Exit code 2 when there are changes (for CI)

```
$ tf plan -detailed-exitcode
```

Wait for the state lock

```
$ tf apply -lock-timeout=5m
```

Run more operations at once

```
$ tf apply -parallelism=20
```

:init — 8 cmds

Backends, provider versions and modules.

Upgrade providers and modules within their constraints

```
$ tf init -upgrade
```

Reconfigure the backend, ignoring the old one

```
$ tf init -reconfigure
```

Move state to a new backend

```
$ tf init -migrate-state
```

Pass backend settings from a file

```
$ tf init -backend-config=backend.hcl
```

List the providers in use

```
$ tf providers
```

Lock provider hashes for several platforms

```
$ tf providers lock -platform=linux_amd64 -
platform=darwin_arm64
```

Update modules

```
$ tf get -update
```

Show the CLI and provider versions

```
$ tf version
```

:state — 8 cmds

What Terraform thinks exists.

List resources in state

```
$ tf state list
```

Show one resource

```
$ tf state show aws_instance.web
```

Show the whole state or a saved plan

```
$ tf show   $ tf show -json tfplan
```

Rename or move a resource

```
$ tf state mv aws_instance.web aws_instance.app
```

Move a resource into a module

```
$ tf state mv aws_instance.web module.app.aws_instance.web
```

Stop managing a resource without destroying it

```
$ tf state rm aws_instance.web
```

Download the remote state

```
$ tf state pull > terraform.tfstate
```

Release a stuck lock

```
$ tf force-unlock LOCK_ID
```

:import

3 cmds

Bring existing infrastructure under Terraform.

Import a resource into state

```
$ tf import aws_instance.web i-0abc123def456
```

Write config for import blocks

```
$ tf plan -generate-config-out=generated.tf
```

Replace a provider source in state

```
$ tf state replace-provider hashicorp/aws registry.example.com/acme/aws
```

:workspaces

5 cmds

Several states for one configuration.

List workspaces

```
$ tf workspace list
```

Show the current workspace

```
$ tf workspace show
```

Create a workspace

```
$ tf workspace new staging
```

Switch workspace

```
$ tf workspace select staging
```

Delete a workspace

```
$ tf workspace delete staging
```

:outputs

5 cmds

Read values back out.

Show all outputs

```
$ tf output
```

Print one output without quotes

```
$ tf output -raw db_endpoint
```

Outputs as JSON

```
$ tf output -json
```

Try expressions against the state

```
$ tf console
```

Draw the dependency graph

```
$ tf graph | dot -Tsvg > graph.svg
```

:code

5 cmds

Keep the configuration clean and correct.

Format files in the current folder

```
$ tf fmt
```

Check formatting everywhere (for CI)

```
$ tf fmt -recursive -check
```

Validate the configuration

```
$ tf validate
```

Run .tftest.hcl tests

```
$ tf test
```

Log in to HCP Terraform

```
$ tf login
```

:debug

5 cmds

Environment variables that change how the CLI behaves.

Debug logs

```
$ TF_LOG=DEBUG tf plan
```

Trace logs to a file

```
$ TF_LOG=TRACE TF_LOG_PATH=./tf.log tf apply
```

Set a variable from the environment

```
$ TF_VAR_region=us-east-1 terraform plan
```

Pick the workspace from the environment

```
$ TF_WORKSPACE=staging tf plan
```

Run in another folder

```
$ tf -chdir=envs/prod plan
```

Updated Nov 8, 2026 · checked against Terraform 1.x and OpenTofu 1.x · sources: Terraform CLI commands, OpenTofu CLI commands

Made by Lucas Bittencourt. Free, no ads, no tracking. Spotted a mistake? Tell me.